PDF Search Engine

TF-IDF · ChromaDB · Three Extraction Backends

Mahmoud Adel · 202210183
Karim Mohamed . 202210006

Natural Language Processing

PROJECT OVERVIEW

What Does This System Do?

Index PDFs

Scan a folder, extract text with the chosen backend, clean with regex, then split into overlapping 500-char chunks — each tagged with its real PDF page number.

TF-IDF Vectors

Fit sklearn's TfidfVectorizer using a custom NLTK tokenizer (lowercase → tokenize → stopword removal → lemmatize). Save the vectorizer to disk for reuse.

Cosine Search

Transform a user query with the saved vectorizer and get from ChromaDB the Top-K most similar chunks using cosine similarity. Results include source file, page number, score, and snippet.

PIPELINE

System Architecture

INDEXING PHASE

SEARCH PHASE

Shared by all 3 backends

`clean_text()` → `chunk_text_with_pages()` → `build_tfidf_vectorizer()` → `index_chunks()`

preprocess_text() used as the TF-IDF tokenizer — tokenize, remove stopwords, lemmatize

EXTRACTION METHODS

Three Extraction Backends

OCR-Based

Tesseract OCR

tesseract_chromadb.py

Renders each page as a 200 DPI image via pdf2image, then runs pytesseract to extract text. Only option for scanned / image-only PDFs.

- ✓ Handles scanned PDFs
- ✓ Works on image-only files
- ✗ Slowest (image + OCR loop)
- ✗ OCR errors possible
- ✗ Needs Tesseract + poppler

★ RECOMMENDED

PyMuPDF (fitz)

fitz_chromadb.py

Reads the PDF's native embedded text layer directly. No image conversion at all — drastically faster and cleaner than OCR.

- ✓ Fastest — ms per page
- ✓ Best text quality
- ✓ PyMuPDF only dep
- ✗ No scanned PDF support

Pure Python

PyPDF

pypdf_chromadb.py

Pure-Python PDF reader with zero system dependencies. Good text extraction quality; portable to any Python environment.

- ✓ Zero system deps
- ✓ Easy cross-platform install
- ✗ Slightly slower than fitz
- ✗ Less layout accuracy

FEATURED BACKEND

PyMuPDF (fitz)

Speed · Quality · Simplicity

Native Text Extraction

Calls fitz.open() then page.get_text() — reads the embedded text stream, zero image rendering.

Milliseconds per Page

Typically 10–50× faster than Tesseract. A 50-page deck is processed in under one second.

Excellent Formatting

Word order and layout come straight from the PDF text layer — minimal OCR artifacts or noise.

Minimal Dependencies

Only pip install pymupdf needed. No Tesseract binary, poppler, or pdf2image installs.

fitz_chromadb.py

```
# extract_text_from_pdf()
def extract_text_from_pdf(
        pdf_path: str):
    """
    Native text extraction.
    Returns [(page_num, text)].
    """
    doc   = fitz.open(pdf_path)
    pages = []

    for i, page in enumerate(
            doc, start=1):
        text = page.get_text()
        if text.strip():
            pages.append(
                (i, clean_text(text)))

    doc.close()
    return pages
```

SHARED PIPELINE

Text Preprocessing & Cleaning

`preprocess_text()`

NLTK pipeline — used as TF-IDF tokenizer

1. **Lowercase**
   `text.lower()`
2. **Tokenize**
   `word_tokenize()`
3. **Remove non-alpha**
   `tok.isalpha()`
4. **Remove stopwords**
   `NLTK english stopwords`
5. **Lemmatize (verb)**
   `lemmatize(tok, pos='v')`

`clean_text()`

Regex cleanup — applied right after PDF extraction

- **Remove URLs**
  `/https?://\S+/`
- **Remove slide numbers**
  `/\b\d+/\d+\b/`
- **Replace bullet symbols**
  `/[•●▸□■–—]/`
- **Fix camelCase splits**
  `/([a-z])([A-Z]) → $1 $2/`
- **Split letter+digit**
  `/([A-Za-z])(\d)/`
- **Remove non-ASCII**
  `/[^\x00-\x7F]+/`

VECTORIZATION

TF-IDF Vectorization

Two-Pass Build Process

1

Pass 1 — Extract & Collect

· Extract text from every new PDF

· Chunk pages (500 chars, 50 overlap)

· Record real page number per chunk

· Record source PDF name per chunk

2

Pass 2 — Vectorize & Store

· Fit TF-IDF on all collected chunks

· Transform → sparse float matrix

· Save vectorizer to .pkl (for queries)

· Store TF-IDF vectors in ChromaDB

TfidfVectorizer — Parameters

```
tokenizer =          preprocess_text
# Full NLTK pipeline

preprocessor =       None
# Tokenizer handles all prep

lowercase =          False
# Lowercased inside tokenizer

min_df =             2
# Drop terms in <2 chunks

max_df =             0.9
# Drop terms in >90% of chunks
```

VECTOR STORE

ChromaDB Integration

Chunk Record in ChromaDB

```
ids:              uuid4()  — auto-generated

embeddings:       TF-IDF float[] vector

documents:        Raw chunk text string

metadata  →:

    source:       "lecture_01.pdf"

    page_number:  7  (real PDF page)

    text:         First 500 chars of chunk
```

Collection Configuration

hnsw:space: "cosine" → cosine distance

embedding_function: None → we supply our own

persist_dir: "./chroma_store"

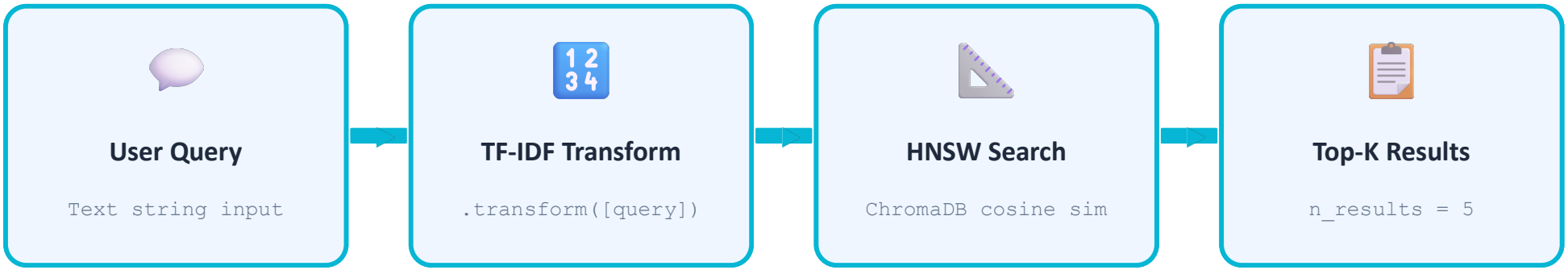
collection: "pdf_tfidf"

Smart Deduplication

- pdf_already_indexed() checks via collection.get()
- Filters by source metadata before processing
- Only new PDFs are extracted and vectorized
- Safe to re-run on a growing PDF folder

RETRIEVAL

Search Mechanism



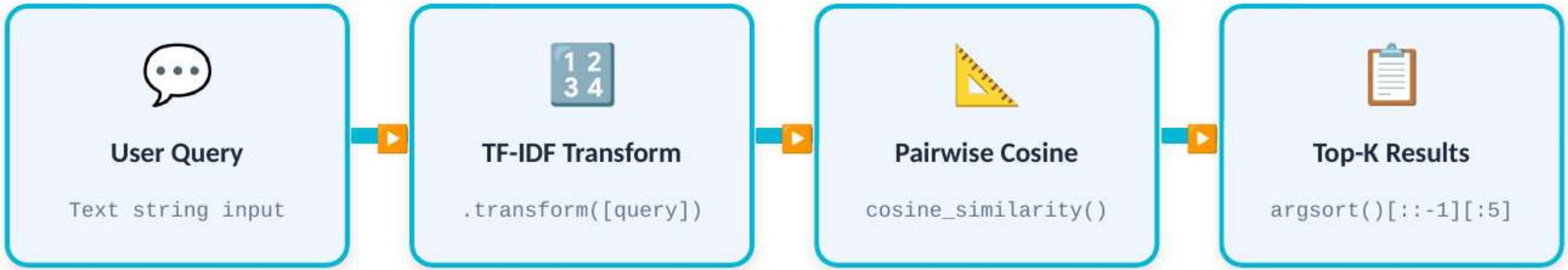
Result format (terminal output)

```
==================================
Query: markov chain transition
==================================

┌─ Result #1 ──────────────────────────────────────
│  File       : lecture_03.pdf
│  Page       : 12        Similarity : 87.4%
│
│  A Markov chain is a stochastic model describing...
└──────────────────────────────────────────────────
```

RETRIEVAL

Search Mechanism



How search() works — sklearn pairwise approach

```
stored = collection.get(
    include=["embeddings","documents","metadatas"])

stored_emb = np.array(stored["embeddings"])

sims = cosine_similarity(query_vec, stored_emb)[0]

top_idx = np.argsort(sims)[::-1][:top_k]
```

1. Fetches ALL embeddings from ChromaDB
2. sklearn computes every pairwise score
3. argsort ranks; slice gives top-K

COMPARISON

Backend Comparison

Feature	Tesseract	⭐ PyMuPDF (fitz)	PyPDF
Approach	OCR on page images	**Native text layer**	Native text layer
Speed	🐢 Slow	⚡ **Very Fast**	🔵 Fast
Text Quality	Variable (OCR errs)	**Excellent**	Good
Scanned PDFs	✓ Yes	✗ **No**	✗ No
Extra Deps	Tesseract + pdf2image	**PyMuPDF only**	pypdf only
Best For	Scanned / image PDFs	**Text-based PDFs** ⭐	Portable, dep-free

OUTPUT SAMPLE

Indexing Process — Terminal Output

```
PS C:\Users\USER\Desktop\files> & C:/Users/USER/anaconda3/envs/XX/python.exe c:/Users/USER/Desk
top/files/fitz_chromadb.py

Found 3 PDF(s) in 'pdfs'.
Connecting to ChromaDB at './chroma_store' ...

Building TF-IDF index (this may take a moment) ...
  [extract] 'backpropagation.pdf' ...
            → 77 chunks across 76 page(s)
  [extract] 'big data mahmoud adel 2022101183.pdf' ...
            → 23 chunks across 7 page(s)
  [extract] 'ch3.pdf' ...
            → 93 chunks across 90 page(s)

  [tfidf] Fitting TF-IDF on 193 chunks ...
C:\Users\USER\anaconda3\envs\XX\lib\site-packages\sklearn\feature_extraction\text.py:517: UserW
arning: The parameter 'token_pattern' will not be used since 'tokenizer' is not None'
  warnings.warn(
  [tfidf] Vocabulary size: 434 terms
  [store] 'backpropagation.pdf' → 77 vectors into ChromaDB ...
  [store] 'big data mahmoud adel 2022101183.pdf' → 23 vectors into ChromaDB ...
  [store] 'ch3.pdf' → 93 vectors into ChromaDB ...
  [done] Index built.

Index ready — 193 total chunk(s) in the collection.

Search engine - Type your query.
Type 'exit' or 'quit' to stop.

Query >
```

OUTPUT SAMPLE

Search Results — Query Output

```
Search engine - Type your query.
Type 'exit' or 'quit' to stop.

Query > markov chain
====================================

Query: markov chain
====================================

┌─ Result #1 ──────────────────────────────────────────────
│  File       : ch3.pdf
│  Page       : 26
│  Similarity : 77.4%
│
│  Markov chain for weather example The graph visualises a Markov chain A Markov chain is a mod
el that defines the probabilities of sequences of random variables (states), which come from a
predefined set E.g., sweather = [s 1=hot, s 2=cold, s 3=warm]
└──────────────────────────────────────────────────────────

┌─ Result #2 ──────────────────────────────────────────────
│  File       : ch3.pdf
│  Page       : 27
│  Similarity : 47.3%
│
│  Markov chain for weather example The chain also defines transition probabilities ai for each
 state i E.g., a 1 = [0.6, 0.1, 0.3] Interpretation: P(s 1=hot s 1=hot) = 0.6 P(s 1=hot s 2=col
d) = 0.1 P(s 1=hot s 3=warm) = 0.3
└──────────────────────────────────────────────────────────
```

KEY TAKEAWAYS

Conclusion

fitz is the Best Default

For text-based PDFs, PyMuPDF (fitz) delivers the fastest extraction and highest text quality. Only one pip install needed.

Lightweight Retrieval

TF-IDF + ChromaDB gives effective document search without large language models, GPUs, or embedding APIs.

Page-Aware Chunking

Every chunk retains its real PDF page number — results pinpoint the exact location in the original document.

Tesseract for Scanned PDFs

When PDFs contain only scanned images with no embedded text, Tesseract OCR is the only extraction option.